

Lab 03

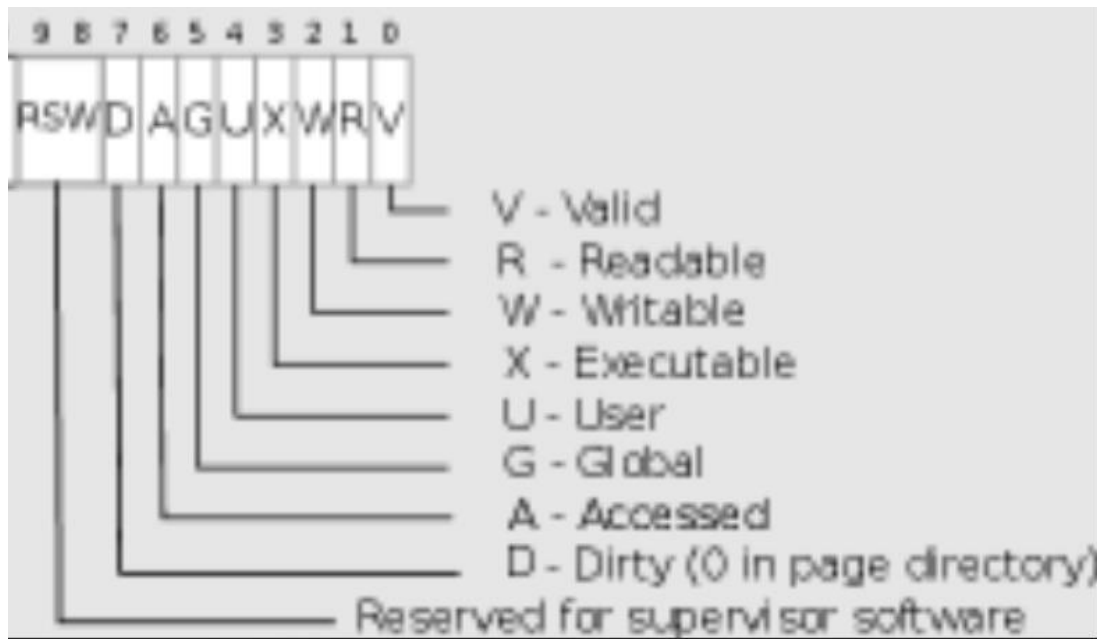
Pagetables

Ayaan Arif Nadeem

30546

TASK 1:

```
$ pgtbltest
print_ptbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C1B pa 0x87F1F000 perm 0x1B
va 0x2000 pte 0x21FC7817 pa 0x87F1E000 perm 0x17
va 0x3000 pte 0x21FC7407 pa 0x87F1D000 perm 0x7
va 0x4000 pte 0x21FC70D7 pa 0x87F1C000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD50C7 pa 0x87F54000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_ptbl: OK
vmprint_test starting
vmprint_test: done
pgaccess_test starting
pgtbltest: pgaccess_test failed: incorrect access bits set, pid=4
```



The `print_pgtbl` output maps out the user process exactly as `xv6` builds it from the ground up. Starting at virtual addresses `0x0` and `0x1000`, the permissions (`0x5B` and `0x1B`) include the Execute and User flags, meaning these pages hold the actual compiled code (text segment) of the program. Right above that at `0x2000` (`0x17`), the Execute flag is swapped for the Write flag, creating the data segment where the program's global variables live. Address `0x3000` (`0x7`) is the clever part: it lacks the User bit entirely, acting as an invisible "guard page" that will safely crash the program if the stack grows too deep and tries to overwrite the data below it. Sitting directly on top of that guard at `0x4000` (`0xD7`) is the user stack itself, which is readable, writable, and user-accessible. Finally, the two pages sitting at the absolute highest virtual addresses (`0x3FFFFFFE000` and `0x3FFFFFFF000`) completely lack the User bit because they belong strictly to the kernel. The writable one (`0xC7`) is the trapframe used to securely save CPU registers during system calls, and the executable one (`0x4B`) is the trampoline page containing the secure code needed to bounce the CPU between user and kernel mode.

TASK 2:

The leaf pages in your `vmprint` output represent the exact same memory segments decoded in the earlier `print_pgtbl` exercise, just visualized hierarchically. Starting at the

bottom of memory, virtual addresses 0x0 and 0x1000 (RXU) hold the program's compiled code, allowing the user to read and execute instructions without being able to accidentally modify them. Right above that, 0x2000 (RWU) is the data segment for global variables, which the user process can read and write but not run as executable code. Address 0x3000 (RW) acts as a protective guard page; it intentionally lacks a User (U) bit so that if the readable and writable user stack directly above it at 0x4000 (RWU) overflows, the hardware instantly triggers a fault before the data segment gets corrupted. Finally, the highest memory addresses are strictly reserved for the kernel and lack User bits: the trapframe at 0x3fffffe000 (RW) securely stores CPU registers during system calls, and the trampoline at 0x3fffff000 (RX) contains the privileged instructions needed to safely transition the processor from user mode into kernel mode.

TASK 4:

getppid() could be sped up by adding a ppid field to struct usyscall. The kernel sets it in kfork and updates it in reparent when a process is adopted by init, so userspace can read the parent pid directly from the read-only USYSCALL page without a kernel crossing. uptime() could work too if the timer interrupt wrote ticks into the page, though it only helps when reads are much more frequent than updates. Syscalls that modify state or require permission checks cannot use the page, because it is read-only to user mode.

TASK 5:

The kernel page table uses S superpage PTEs (2MB leaves at level 1) and N regular 4KB leaf PTEs. Each superpage replaces a full bottom-level page-table page of 512 PTEs, so the S superpages save S page-table pages (4*S KB), plus 511 TLB entries' worth of reach each.

```
== Test pgtbltest ==
$ make qemu-gdb
(10.7s)
== Test   pgtbltest: vmprint_test ==
pgtbltest: vmprint_test: OK
== Test   pgtbltest: pgaccess ==
pgtbltest: pgaccess: OK
== Test   pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
== Test   pgtbltest: superpg ==
pgtbltest: superpg: OK
== Test answers-pgtbl.txt ==
answers-pgtbl.txt: OK
== Test usertests ==
$ make qemu-gdb
```